

Demo Script

A condensed, copy-paste-ready run sheet for the live walkthrough — same content as [docs/WALKTHROUGH.md](#), formatted for scanning quickly while presenting or following along. About 10 minutes end to end.

* **On a corporate network?** If GitHub is blocked, see [Corporate network access](#) at the very end of this document before starting.

Before starting

- **Fork the repo, then clone your fork** — not this one. Cloning `alfredorueda/ci-demo-python` directly won't let you push branches or open pull requests, since you don't have write access to it:
 1. On GitHub, go to `https://github.com/alfredorueda/ci-demo-python` and click **Fork** (top-right).
 2. Clone *your* fork, replacing `<your-username>` :

```
git clone https://github.com/<your-username>/ci-demo-python.git
cd ci-demo-python
```

Every command below runs inside your fork.

- **Install dependencies** — `pytest` is not installed by default, it comes from `requirements.txt` :

```
python3 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Confirmed it worked if the terminal prompt now starts with `(.venv)` and `pytest --version` prints a version instead of "command not found".

- On `main`, terminal open, the repository's **Actions** tab open in a browser tab.
- Confirm the baseline:

```
git checkout main
git pull
pytest -v
```

Expected: `15 passed` .

1. Baseline (~1 min)

```
pytest -v
```

15 passed in ~0.02s — no Docker, no database, no network call. That's what makes the rest of this run fast.

2. Branch, then break the build on purpose (~2 min)

```
git checkout -b break-the-build
```

In `portfolio_domain/portfolio.py`, inside `buy(...)`, comment out:

```
self._cash = self._cash.subtract(cost)
```

```
pytest -v
```

Expected: 2 failed, 13 passed — `test_buy_deducts_cash_balance` and `test_sell_increases_cash_balance`. One bug, two broken behaviors: this is why a suite covers more than one scenario.

3. Push and open a pull request (~2 min)

```
git add -A
git commit -m "Introduce a bug on purpose for the CI demo"
git push -u origin break-the-build
gh pr create --fill
```

(No `gh`? Use the "Compare & pull request" banner GitHub shows on the repository page after the push.)

4. Watch the check fail (~2 min)

- Refresh the PR page: the check goes yellow (queued/running) → red, in about 10–20 seconds.
- **Details** on the failed check → same `pytest` output as local.
- **Merge** button: greyed out — *"Merging is blocked — Required statuses must pass before merging."* Applies to everyone, including repository admins — that's the branch protection rule on `main`.

5. Fix it and watch it go green (~2 min)

Uncomment the line:

```
self._cash = self._cash.subtract(cost)
```

```
pytest -v
git add -A
git commit -m "Fix the cash balance bug"
git push
```

Refresh the PR: check turns green → **Merge** button active → merge (`Squash and merge` is a fine default).

6. Recap (~1–2 min)

- This whole loop took minutes only because the suite is fast and infrastructure-free — a slow one trains people to stop waiting for it, which is how bugs like this reach `main` on real projects.
- Branch protection turns "please run the tests before merging" into "you cannot merge until they pass" — enforced, not requested.
- The entire pipeline is `.github/workflows/ci.yml` — versioned and reviewed like any other code.

If something stalls

- Actions runs typically take 10–20s to go from "queued" to a result — that gap is normal, not a failure.
- Branch protection can be turned off from **Settings** → **Branches** if `main` needs to be unblocked for any reason.
- `pytest: command not found`? The virtual environment isn't active — re-run `source .venv/bin/activate` (macOS/Linux) or `.venv\Scripts\Activate.ps1` (Windows) from the "Install dependencies" step above.

For every step explained in full, with diagrams, see [docs/WALKTHROUGH.md](#). To repeat this same exercise on your own fork afterwards, see its "Try it yourself" section.

Corporate network access

If GitHub is blocked on your corporate network, `git clone`, `git push`, and friends may fail with a connection or timeout error. This is a network-level restriction, unrelated to this repository or this walkthrough — check with your organization: many companies already document the exact proxy configuration needed for `git` /GitHub in an internal channel (for example, shared over Teams). Ask your IT/network team or your training organizer if you're not sure where to find it.